Navya: complete project record

Autonomous car on a TZ-306 ride-on toy. Everything done from 29 October to 9 November 2026: the car, every connection, all the code, every test with its result, every problem and how it was fixed, where we are stuck now, and what is left.

Owner: Sujal Negi. Repository: github.com/sujal128005/navya_autonomouscar (branch main). Local copy on the laptop: C:\Users\connt\dev\navya_autonomouscar. This PDF is made by docs/report/make_full_record.py.

WHERE WE ARE STUCK (9 Nov 2026)
The front steering does not move yet. We now drive the toy's steering motor through the **2-channel relay module**. The new firmware is on the Arduino and works: on the first test (front lifted, 1-second pulse) **relay 1 clicked, but the motor did not move**. So the Arduino and the relay coil are fine, and the 12 V path through the relay contacts to the motor is not complete. Next step: check the relay screw terminals (photo), battery + connection and fuse, then test again. Details in section 10.

WHAT WORKS (tested on the real car)
Demo 1 shown on 30 Oct (drive, slow, stop for an obstacle, drive on by itself). Rear drive with both hub motors, manual and AUTO modes, radio emergency stop, laptop-silent stop (0.26 s), obstacle stop on the ground (rests 1.10 m from a box), person detection on the ground (rests 1.8-2.0 m away), trip recorder, camera and LiDAR calibration. Jetson Orin Nano set up and bench-tested on the car (3 Nov). The steering motor, gearbox, grey battery and fuse are proven good.

Contents

1. The project in short
2. The hardware
3. How the system works
4. Day-by-day record (29 Oct to 9 Nov)
5. All connections (wiring)
6. The steering story: every test and result
7. Firmware (Arduino) explained
8. Software (laptop / Jetson) explained, with commands
9. Every problem found and how it was fixed
10. Where we are stuck now, and how to get unstuck
11. What is left (full to-do list)
12. Parts, bill and what we own
13. Rules we work by
14. Appendix A: relay firmware change (exact diff)
15. Appendix B: config.py (all settings)
16. Appendix C: steering test script
17. Appendix D: full Arduino firmware

1. The project in short

Navya is a low-cost autonomous car built on a TZ-306 4x4 kids' ride-on car. (It is not *Navyam*, which is a separate drone project.) The long-term goal is a car that drives itself like a Tesla, on a student budget. We build it in stages:

Stage	Goal	Status on 9 Nov
Demo 1	Drive straight; slow down and stop for an obstacle; drive on by itself when it is removed	**DONE, shown 30 Oct 2026**
Object detection	Recognise people and stop signs; stop further from people	**Tested on the ground 2 Nov**
Onboard computer	Jetson Orin Nano on the car, laptop only as the screen	**Set up and bench-tested 3 Nov; needs a 12 V step-down converter to drive**
Demo 2: steering	Turn the front wheels from the Arduino, then follow the corridor between the walls (lane following)	**STUCK: steering motor does not move yet**
Later	LiDAR mapping, wheel-speed feedback (needs an Arduino Mega)	Planned

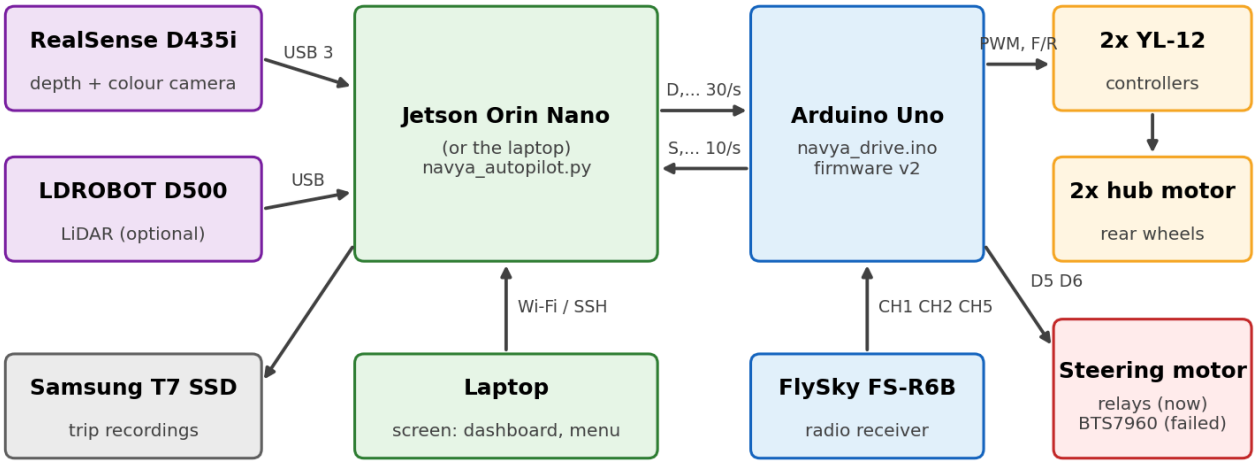
The professor's rule for steering: keep the toy's own DC steering motor and gearbox ("not to change steering, do work with current only"). No servo. We only add a driver for the motor (and later a position sensor).

2. The hardware

Part	Details	Role
TZ-306 4x4 ride-on car	about 31 kg with everything, wheelbase 0.745 m, width ~0.62 m	chassis
2x 8" BLDC hub motors	24 V 350 W, with drum brake arms, on welded steel brackets	rear drive (replaced the toy's rear wheels)
2x YL-12 brushless sine-wave controllers	24 V, 350 W, 16 A, brake active LOW, under-voltage cut-off 21 V	one per hub motor
2x 6S LiPo	22.2 V 5200 mAh (25.2 V full)	motor power, one per controller
Grey toy battery	6-FMD-8, 12 V 8 Ah sealed lead-acid (SLA)	steering motor power
Arduino Uno (CH340 clone)	16 MHz, USB serial at 250000 baud	radio input, throttle, brakes, steering, safety rules
FlySky FS-CT6B + FS-R6B	6-channel transmitter and receiver	manual driving, MANUAL/AUTO switch (CH5), emergency stop (stick back)
Intel RealSense D435i	depth + colour, at the front, 0.554 m high, 4.4 deg down	obstacles, walls, object detection
LDROBOT D500 LiDAR (STL-19P)	360 deg, USB via CP210x, 230400 baud, on the dash bar	second obstacle sensor (optional)
NVIDIA Jetson Orin Nano Super	JetPack 6.2, L4T 36.4.7, user nexara9, MAXN_SUPER	onboard computer since 3 Nov
Samsung T7 SSD 1 TB	mounted at /mnt/t7ssd on the Jetson	trip recordings
Windows laptop	i7-1260P, Iris Xe, Python 3.14	development and the screen
Toy steering DC motor + gearbox	two wires: dark blue and brown	turns the front wheels
Toy's blue control board	HONGHUIDIANZI HH-678K-RXSN2	original toy electronics; not used for steering, NOT modified
Steering driver (history)	L298N (too weak) -> SmartElex BTS7960 (dead) -> 2-channel 5 V relay module (now)	lets the Arduino turn the steering motor both ways
Parts bought for steering (arrived 7 Nov)	BTS7960, 2x 10k pot, 2x KW4 limit switch, inline blade fuse holders, 4x Littelfuse ATOF 10 A 32 V, 18 AWG red + black wire	Robu.in order, Rs 2,302
Also owned, unused	Raspberry Pi 3B, ESP32, L298N, one unidentified LiDAR	

Not owned: a multimeter (strongly needed; see section 11).

3. How the system works



Chain of command. The onboard computer (Jetson, or the laptop) reads the camera (and LiDAR), decides a speed and a steering angle, and sends the Arduino one short line about 30 times a second, for example `D,500,500,0,0` (left speed, right speed, brake, steering). The Arduino replies 10 times a second with a status line `S,...` (17 fields). The Arduino turns the speed into throttle PWM for the two YL-12 controllers, drives the brake wires, and drives the steering motor.

Driving modes (CH5 switch on the transmitter)

Mode	What happens
MANUAL	the right stick drives: up = forward, back = brake; left/right = steering
AUTO (CH5 above 1700 us)	the computer drives and steers. Pulling the stick back still stops the car
NO_RC	no radio signal for 0.1 s: motors off, brakes on, steering off

Safety layers

Layer	Protection	Tested
Person with the transmitter	stick back = stop + brake in any mode, even while the computer says drive	**yes, 2 Nov (brake on in all 283 readings)**
Arduino	stops if the radio is lost for 0.1 s or the computer is silent for 0.3 s	**yes, motors off after 0.26 s**
Arduino, steering	stops the steering motor at a limit switch, after 1.5 s without pot movement (stall), on a broken pot wire, with no radio	firmware only
Computer	stops if depth frames stop for 0.5 s, if the LiDAR goes quiet (with --lidar), or if the lens is covered	**yes**
Battery monitor	warning at 22.2 V; below 21.6 V for 3 s it stops and stays stopped	**needs 4 resistors (not fitted)**
Pre-flight check	checks every part before a run	**yes**

Obstacle detection and speed

Each depth image becomes 3D points. Only points inside a box in front of the car count: 0.45 m either side of the centre line, from 0.15 m above the floor up to 1.6 m, up to 4.5 m ahead. At least 800 points are needed. The nearest 5% of those points is the obstacle distance.

Distance to obstacle	What the car does
more than 3.8 m	CLEAR: cruise (speed 130)
3.8 m to 2.0 m	SLOWING: speed falls smoothly to creep (80)

2.0 m or less	STOPPED: brake. The 31 kg car rolls ~0.85 m more, so it rests ~1.1 m away
nothing within 2.4 m for 0.5 s	RESUMING: drives on, with a 0.4 s start boost (250)

With `--vision`, people make the car stop 1 m earlier (3.0 m). A stop sign within 2.5 m: stop, wait 3 s, drive on, ignore stop signs for 5 s. Detection and LiDAR can only make the car more careful; the depth check always runs.

4. Day-by-day record

29 Oct 2026

- Project started. Hardware identified from photos. Demo 1 goal set.
- Old work in the repository deleted (commit 4236da6), as the user asked.
- Demo 1 code written and pushed (ea16343): Arduino firmware (D9/D10 throttle, D8/D7 F/R, brake, radio CH2 on D3 and CH5 on D2) and the laptop autopilot.
- Firmware uploaded. The CH340 clone needed its USB driver rolled back to 3.5.2019.1 (version 3.9 fails with "cannot set com-state").
- Bench test: wheels start at PWM about 75. PWM_START = 75, PWM_MAX = 130 (aafcb04).
- First indoor AUTO ground run WORKED: drive, slow, stop, drive on. The car coasted ~0.9 m past the stop (no brake wire), so stop distance raised to 2.0 m, slow-down 3.8 m, cruise 130 (02378bd). Camera calibrated 0.555 m / 12.2 deg.

30 Oct 2026 - DEMO 1 SHOWN

- Corridor viewer (live lane, bird's-eye map, 3D point cloud) and lane finder added.
- Tape-strip lane detection failed on the shiny granite floor (mirror reflections), so the lane is found from the walls with depth (LANE_SOURCE = "walls").
- Demo 1 shown with the phone view over the laptop's hotspot (http://192.168.137.1:8080). Tweaks: resume delay 0.5 s, start boost 250 for 0.4 s.

31 Oct 2026

- Object detection: YOLO11n. Then OpenVINO on the Intel GPU: 19 ms per image (ONNX 38 ms fallback). --vision: stop 1 m earlier for people; stop-sign rule.
- Trip recorder (--record), raw recording and replay, trip report.
- 128 GB SD card (old Jetson card) wiped with the user's approval and formatted "NAVYA" for recordings.
- Steering parts studied. A 180 kg-cm servo was first chosen, then the professor decided: keep the toy steering motor. Final Robu.in cart: 7 lines, Rs 2,302 (BTS7960, 2 pots, 2 limit switches, fuse holders, 10 A fuses, red and black wire). The user paid themselves.

1 Nov 2026

- Firmware v2 (b1c368f): steering position control (BTS7960 on D5/D6, enable A3, pot A0, limit switches A1/A2, radio CH1 on D4). Brakes MOVED to D11/D12.
- steering_calibrate.py, lane_follow.py (pure pursuit), --steer.
- Pre-flight check, live tuning sliders, battery monitor (A4/A5), Ask Navya.
- Ask Navya changed to free AI: Gemini first, Groq fallback (no Anthropic key, as the user asked). Keys go in autopilot/keys.env (git-ignored).
- Navya.bat menu, GitHub Actions checks, LiDAR code, project report. Review fixes (2db5de0).

2 Nov 2026 - hardware test day

- Firmware v2 on the car. The CH340 3.9 driver package was deleted from Windows.
- Serial link moved to 250000 baud (115200 lost ~15 bytes a minute).
- Stick-back stop line lowered to 1300 us (stick centre had drifted to ~1410).
- LiDAR mounted above the camera and aligned (-1 deg, clockwise, 0.365 m, 10 cm left).
- Camera re-mounted rigidly: 0.554 m, 4.4 deg down.
- Ground run: braked at 1.97 m, rested 1.10 m from the box.
- --vision ground run: person seen at 5 m, braked 2.85 m, rested 1.8-2.0 m away.
- The laptop battery hit 4% and slept: pre-flight now fails below 20%.

3 Nov 2026 - Jetson

- Jetson Orin Nano Super set up: own Python environment, GPU PyTorch, TensorRT detection 23 ms per image, CH340 driver built for the JetPack kernel.
- Samsung T7 SSD erased (user ran the script; drone data deliberately discarded) and mounted at /mnt/t7ssd for recordings.
- Bench test on the car (wall power): camera, Arduino, LiDAR, detection, recording, dashboard all together; LiDAR 0 times "lost" (laptop had ~6 a minute).
- Navya Jetson.bat: one double-click on the laptop finds the Jetson and opens its menu.
- Wiring document docs/Navya_wiring.pdf (0bed771).

4 Nov 2026 - first steering test (L298N)

- Advice: one battery or two for the controllers (one is enough, the second as a swap).
- Brake wire question: the brake plug has yellow and orange wires and 2 sockets (one extended). Not yet wired.
- Wiring image Navya_wiring_16x9.png pushed (8eb2a54).
- L298N steering test: NO movement. Loose jumper wires at the Arduino kept dropping the radio; the Arduino was rewired without the breadboard.

5 Nov 2026 - L298N works, but too weak

- The user's old working code drove ENA from D10. The fix: remove the ENB jumper and drive ENB from A3. Then, front lifted, 255 for 1.4 s MOVED the wheels (0.8 s gave only a sound).
- On the ground: centre -> full right in 2 pulses; full right -> centre in 1 pulse; centre -> left: only a hum.
- Straight from the battery the motor took the wheels full right -> full left in 3 touches. So motor and battery are good; the L298N loses too much voltage.
- Grey battery charged with the iMAX B6AC (Pb mode, 12.0V(6P), 0.8 A). It finished at 14.40 V / 0 mAh: it was already full.
- Decision: keep the BTS7960 order. L298N only for lifted tests.

6 Nov 2026 (side task, not Navya)

- Answers written for the Claude open-source programme application (reach and impact, how the subscription would be used, other info).

7 Nov 2026 - parts arrive, BTS7960 fails

- Parts arrived. Fuse holder explained; the fuse had been soldered straight to wires: use the inline holder. B- wire was red: use black, tape bare ends.
- Receiver to Arduino wiring repeated. Windows put CH340 driver 3.9 back on a new USB port (error 31); rolled back to 3.5.2019.1. Arduino became COM13.
- BTS7960: header and screw terminals came UNSOLDERED. The user soldered them; a solder bridge on L_EN shorted 5V (the 5V wire got warm), then it was fixed.
- No motor movement in either direction, with the firmware and with a bench test (EN and RPWM/LPWM straight to 5V). The board is presumed damaged.
- The fuse (Littelfuse ATOF 0287010, 10 A 32 V) has no indicator light; it is good (the motor ran through it straight from the battery). Datasheets checked.

8-9 Nov 2026 - potentiometer, toy board, relays

- How to fit the potentiometer: diagrams, 3D view, marked photo, YouTube links. From the underside video: the steering turns around a cross pin under the gearbox; a black arm goes from there to the tie rod. Option A: pot under the centre, on a plate held by the 2 gearbox screws. Option B: wire linkage from the tie rod. Coupler ideas: pot knob, M-seal, pen tube, bottle cap.

- The steering column is too thick for the pot, and it only turns the wheels by hand; the motor turns them in remote mode.
- The user: mechanical changes are difficult now. So: steering without a pot (open-loop: find the ends by stall, centre by time, correct with the camera).
- Toy board reviewed (HH-678K-RXSN2: MCU with 16 MHz crystal, GOLDEN GH-1C-12H 15 A relay, 2 MOSFETs, red 7-pin connector + - F B L S). Decision: do not modify it.
- Chosen: drive the steering motor with the 2-channel 5 V relay module (H-bridge). Firmware relay mode written (STEER_RELAYS), compiled (9684 bytes).
- Upload: Arduino on COM14, driver 3.9 again; user rolled back to 3.5.2019.1; upload OK. CFG shows steer_relays=1, radio in AUTO.
- Test, front lifted, 1 s pulse on relay 1: relay CLICKED, motor did NOT move. **We are here.**

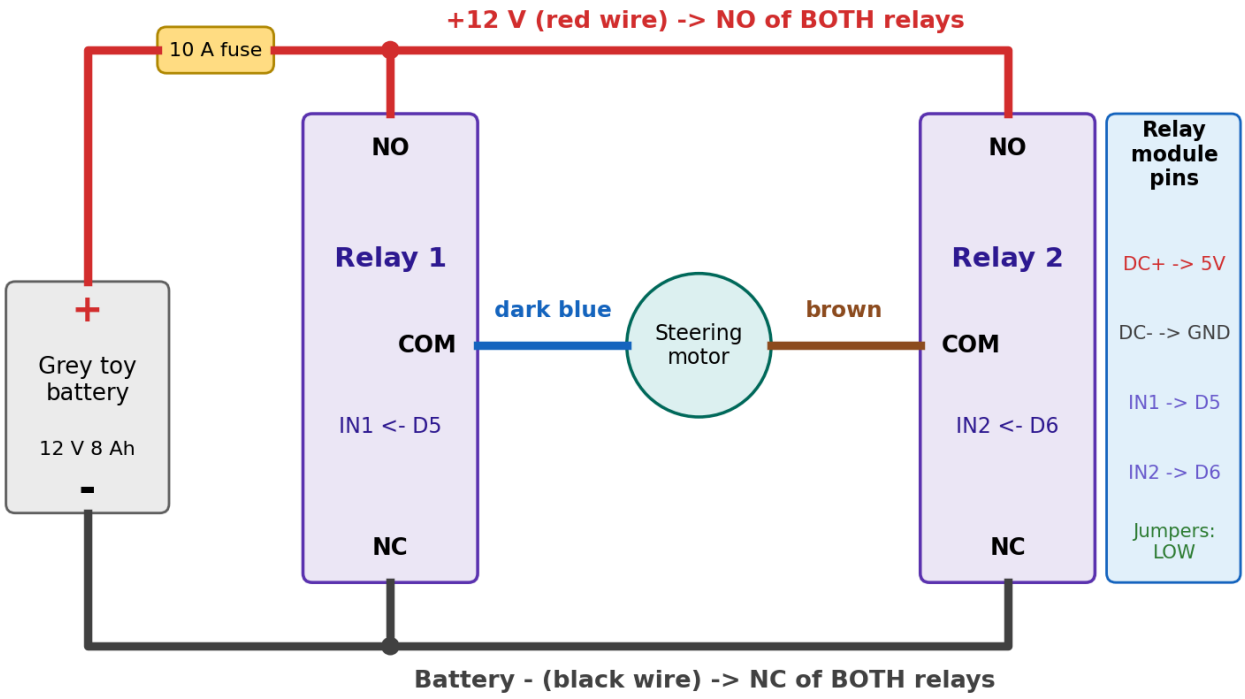
5. All connections (wiring)

5.1 Arduino Uno pins (firmware v2)

Arduino pin	Goes to	Status
D9	LEFT controller throttle signal (purple wire)	**working**
D10	RIGHT controller throttle signal (purple wire)	**working**
D8	LEFT controller F/R (silver wire)	**working**
D7	RIGHT controller F/R (silver wire)	**working**
GND	BOTH controllers' throttle black + F/R black wires (shared ground)	**working**
D11	LEFT controller brake wire (active LOW)	**not wired yet**
D12	RIGHT controller brake wire (active LOW)	**not wired yet**
D2	receiver CH5 signal (MANUAL/AUTO switch)	**working**
D3	receiver CH2 signal (right stick up/down)	**working**
D4	receiver CH1 signal (right stick left/right = manual steering)	**working**
5V / GND	receiver + / - (middle / outer pin of any channel)	**working**
D5	relay module IN1 (relay 1)	**wired 9 Nov, motor not moving yet**
D6	relay module IN2 (relay 2)	**wired 9 Nov**
5V / GND	relay module DC+ / DC-	**relay clicks**
A3	steering enable (only for BTS7960 / L298N; leave EMPTY with relays)	not used now
A0	potentiometer middle pin (outer pins to 5V and GND)	**pot not fitted**
A1 / A2	left / right limit switch to GND (closed = at that lock)	**not fitted**
A4 / A5	left / right LiPo through 47k / 10k divider (battery monitor)	**not fitted**
USB	laptop or Jetson	**working (now COM14 on the laptop)**

Every Uno pin is now in use. More inputs (wheel-speed sensors) need an Arduino Mega.

5.2 Steering: relay module (CURRENT setup, 9 Nov)



Both relays OFF = both motor wires on battery - -> motor stopped. Relay 1 ON = dark blue gets +12 V -> turns one way. Relay 2 ON = brown gets +12 V -> turns the other way. Both ON = both on +12 V -> stopped (no short circuit possible).

Relay module	Connect to
DC+ (VCC)	Arduino 5V
DC- (GND)	Arduino GND
IN1	Arduino D5
IN2	Arduino D6
Trigger jumpers (both)	on LOW (input pulled LOW = relay ON)
Relay 1 COM	steering motor DARK BLUE wire
Relay 2 COM	steering motor BROWN wire
Relay 1 NO and Relay 2 NO (joined)	+12 V from the battery through the 10 A fuse
Relay 1 NC and Relay 2 NC (joined)	battery - (negative)

D5 (relay 1)	D6 (relay 2)	Motor
off	off	stopped (both wires on battery -)
ON	off	turns one way (dark blue +12 V)
off	ON	turns the other way (brown +12 V)
ON	ON	stopped (both on +12 V); this can never short the battery

Relay contacts are rated 10 A at 30 V DC. Use 18 AWG wire for battery and motor. The old driver (BTS7960 / L298N) must be completely disconnected. Relays give full power or nothing: no speed control (no PWM).

5.3 Rear drive: YL-12 controller plugs

Plug	Wire colours
Throttle	black = GND, orange = +5 V out (leave unconnected), purple = signal
F/R (direction)	black = GND, silver = signal (open = forward)

Brake (optional)	"Brake: Low": pulling the brake wire to GND brakes. On our car the plug has yellow and orange wires and 2 sockets (one extended). Not wired yet.

Power

- One 6S LiPo per controller. Join the two battery negatives with a proper wire, so motor current never flows through the Arduino's ground.
- Full charge 25.2 V; the controllers cut out at 21 V.
- Power the Arduino before (or with) the controllers: they refuse to start if they see throttle at power-on.
- Steering motor: from the grey 12 V battery through the 10 A inline fuse.
- Recommended: a 10k resistor from each purple throttle wire to GND, and a kill switch on the controllers' power-lock wires.

5.4 Earlier steering drivers (for the record)

L298N (tested 4-5 Nov)

L298N	Connect to
IN3 / IN4	Arduino D5 / D6
ENB	Arduino A3 (jumper REMOVED; the ENB pin in line with the IN pins)
OUT3 / OUT4	steering motor dark blue / brown
12V / GND	grey battery + / -
GND	also to Arduino GND

BTS7960 (tested 7 Nov, presumed dead)

BTS7960 pin	Connect to
VCC / GND	Arduino 5V / GND
RPWM / LPWM	Arduino D5 / D6
R_EN + L_EN	Arduino A3 (or 5V)
R_IS / L_IS	not used
B+ / B-	12 V battery through the 10 A fuse / battery -
M+ / M-	steering motor wires

Header labels: top row VCC, R_IS, R_EN, RPWM; bottom row GND, L_IS, L_EN, LPWM.

5.5 Battery monitor (optional, not fitted)

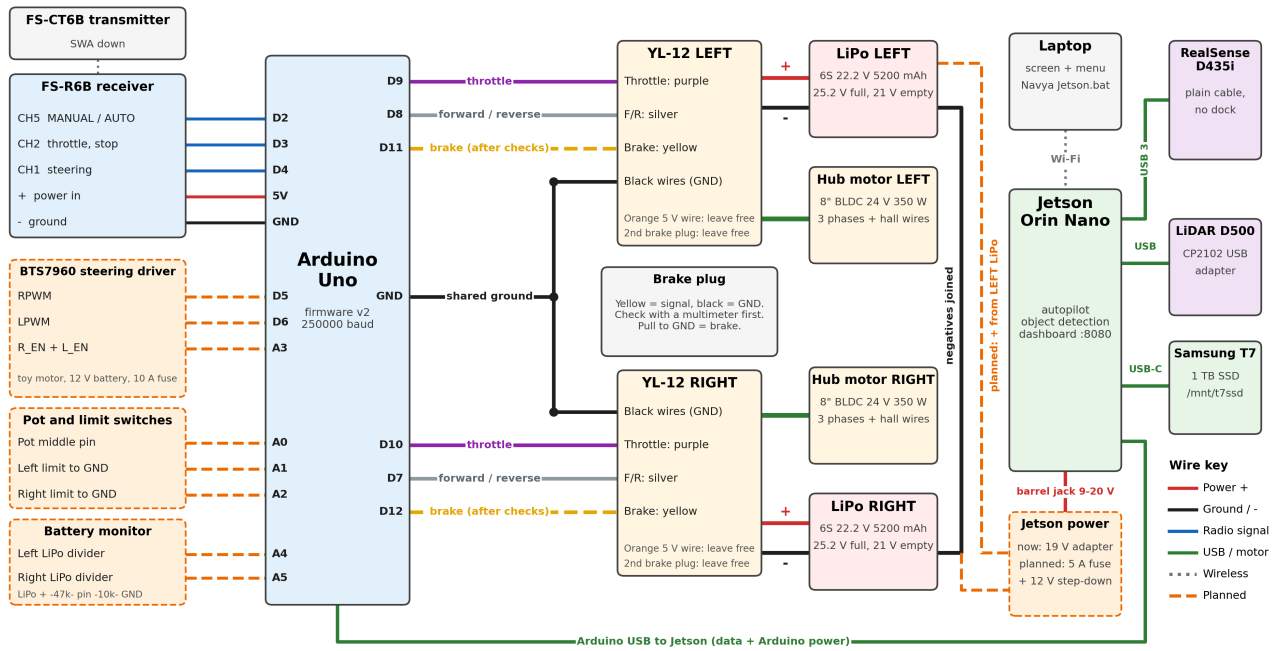
For each LiPo: **LiPo + -> 47 kOhm -> Uno pin -> 10 kOhm -> GND**. Left battery A4, right battery A5. 25.2 V reads as 4.4 V at the pin.

5.6 Computer connections

Device	Connection
RealSense D435i	USB 3 cable straight into the Jetson (or laptop). Not through a USB-C dock or cheap hub
Arduino	USB (CH340). Jetson: needs jetson/build_ch341.sh. Laptop: driver 3.5.2019.1
LiDAR	USB via its CP210x adapter
Samsung T7	Jetson USB-C, plug in before switching on (or sudo mount /mnt/t7ssd)
Laptop <-> Jetson	same Wi-Fi (phone hotspot or laptop hotspot; college Wi-Fi blocks it). ssh jetson, key ~/.ssh/navya_jetson

5.7 Wiring image of 8 Nov (before the relay change)

docs/Navya_wiring_16x9.png. The steering part of this image is replaced by section 5.2.



6. The steering story: every test and result

The toy's steering motor (two wires, dark blue and brown) turns the front wheels through the toy gearbox. The Arduino cannot power a motor directly, so it needs a **driver** that can switch 12 V both ways. This is everything we tried.

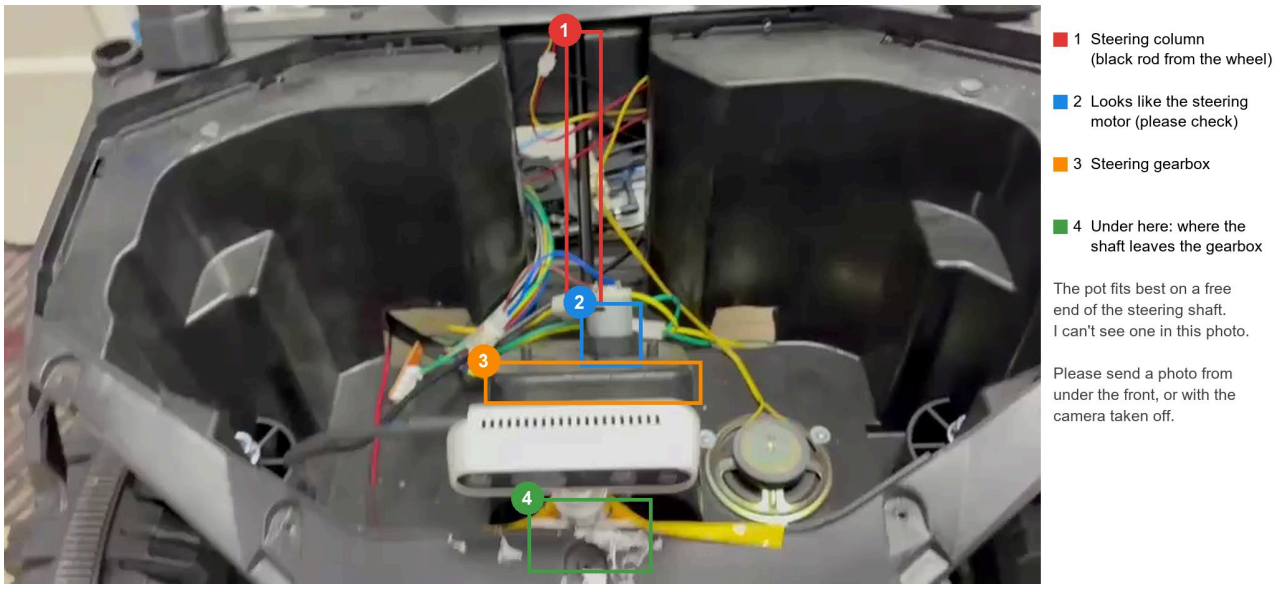
Date	Setup	Test	Result
4 Nov	L298N, ENB jumper on	full-power pulses, wheels lifted	**no movement**
4 Nov	same, IN3/IN4 and OUT3/ OUT4	retests after fixing loose wires	**no movement**
5 Nov	L298N, ENB driven by A3 (jumper removed)	lifted, 255 for 0.8 s	sound only
5 Nov	same	lifted, 255 for 1.4 s	**moved**
5 Nov	same	lifted, centre -> left	moved about half way
5 Nov	same, ON THE GROUND	centre -> full right	**2 pulses**
5 Nov	same, ground	full right -> centre	**1 pulse**
5 Nov	same, ground	centre -> left	**hum only, no movement**
5 Nov	motor straight on the battery, ground	full right -> full left	**3 short touches: motor and battery GOOD**
5 Nov	verdict		L298N loses too much voltage (about 2-3 V)
7 Nov	BTS7960 (after soldering)	firmware pulses both ways	**no movement**
7 Nov	BTS7960 bench bypass	EN to 5V, RPWM or LPWM to 5V	**no movement**
7 Nov	verdict		BTS7960 presumed damaged (unsoldered from the factory, then a 5 V short on L_EN)
9 Nov	2-channel relay module (H-bridge)	lifted, relay 1 on for 1.0 s	**relay clicked, motor did not move** - see section 10

6.1 Why relays

- The user already owns the relay module; no new parts.
- A relay contact loses almost no voltage, unlike the L298N, so the motor gets the full 12 V (the same as touching the battery, which worked).
- Downside: full power or off, no speed control, and a click every switch. Fine for steering.
- Wired as an H-bridge where both relays off (or both on) means stopped, so a mistake in the code can never short the battery.

6.2 Fitting the potentiometer (postponed)

Without a position sensor the Arduino does not know where the wheels are. The plan is a 10k pot that turns with the steering:



Marked photo used while looking for the pot location (early guess; numbers 1-4 as labelled).

Option	How	Notes
A: under the centre (coaxial)	pot under the cross pin of the steering, body held by a small plate on the 2 gearbox screws; shaft coupled to the pin	couplers: pot knob, M-seal putty, pen tube, bottle cap (pot shaft is 6 mm knurled, 18 splines)
B: linkage	pot on the chassis, a stiff wire from the tie-rod tab to a lever on the pot	no need to reach the centre
Not possible	on the steering column	the column is too thick and is only for manual steering; the motor turns the wheels in remote mode

Decision (8-9 Nov): mechanical changes are difficult for now, so steer without the pot first.

6.3 Steering without a pot (plan, code not written yet)

Step	How
Find the ends	drive the motor until it stalls at full left / full right (stall = end)
Find the centre	from full right, drive left for half the measured end-to-end time
While driving	short pulses left/right; the camera's lane position corrects the error
Limit	firmware stall cut-off 1.5 s; without a pot, single pulses are capped at 1.4 s

6.4 The toy's own board

HONGHUIDIANZI HH-678K-RXSN2: a microcontroller with a 16 MHz crystal, one GOLDEN GH-1C-12H 15 A relay, two TO-220 MOSFETs, a red 7-pin connector marked roughly + - F B L S (drive inputs). We decided **not** to modify it: there is no documentation, and our own relay module does the same job safely.

7. Firmware (Arduino) explained

File: `firmware/navya_drive/navya_drive.ino` (firmware v2, 656 lines; full text in Appendix D). Board: Arduino Uno, uploaded with arduino-cli (FQBN arduino:avr:uno).

Commands the computer can send (250000 baud, one per line)

Command	Meaning
D,<left>,<right>,<brake>[,<steer>]	speed 0..1000 per wheel, brake 0/1, steering -1000 (full left)..1000 (full right)
R,<pwmLeft>,<pwmRight>	raw throttle PWM (calibration only)
M,<pwm>	raw steering motor -255..255 (AUTO only; used for our steering tests)
K,L / K,C / K,R	store the pot reading as full left / straight / full right (EEPROM)
K,I,<0|1>	1 = RPWM turns the wheels left
K,X	forget the steering calibration
?	print the settings (CFG line)

Status line the Arduino sends 10 times a second (17 fields)

`S,mode,ch2_us,ch5_us,host_ok,pwmLeft,pwmRight,brake,ch1_us,steer_target,steer_pos,`
`pot_raw,limit_left,limit_right,steer_state,battery_left_mV,battery_right_mV`

Example from 9 Nov: S,AUTO,1512,2008,0,0,0,1,1500,0,0,969,0,0,NOCAL,0,0 (AUTO, stick centred, CH5 at 2008 us, brake on, steering not calibrated, no battery dividers).

Key settings in the firmware

Setting	Value	Meaning
PWM_START / PWM_MAX	75 / 130	wheels start at 75; 130 = top speed cap
RAMP_UP_PER_10MS	5	0 to full speed in 2 s
RC_AUTO_ABOVE_US	1700	CH5 above this = AUTO
RC_STOP_BELOW_US	1300	stick below this = stop + brake
RC_TIMEOUT_MS / HOST_TIMEOUT_MS	100 / 300	radio lost / computer silent -> stop
STEER_STALL_MS	1500	steering driven 1.5 s without pot movement = stall
STEER_RELAYS	true	NEW 9 Nov: relay module instead of BTS7960
STEER_RELAY_ON_PWM	40	|pwm| from 40 switches a relay on
STEER_RELAY_MIN_MS / GAP_MS	60 / 100	no chatter; a reversal goes through "both off" for 100 ms

The relay change (9 Nov, NOT committed yet)

- New function `steerRelays()`: D5 LOW = relay 1 on, D6 LOW = relay 2 on, pin released (floating) = relay off. Never both on at once.
- Both relays are OFF at start-up and whenever the Arduino resets.
- `steerOff()` and `steerMotorRaw()` use the relays when STEER_RELAYS is true; the old BTS7960 code stays for STEER_RELAYS = false.
- The CFG line now shows `steer_relays=1`.
- Compiled: 9684 bytes flash (30%), 379 bytes RAM (18%). Uploaded to COM14 on 9 Nov.

How to upload

```
arduino-cli compile --upload -p COM14 --fqbn arduino:avr:uno firmware/navya_drive
(arduino-cli.exe is in C:\Users\connt\AppData\Local\Programs\Arduino IDE\resources\app\lib\backend\resources\)
```

Close the Arduino IDE Serial Monitor first. If the upload says "cannot set com-state": Device Manager -> Ports -> USB-SERIAL CH340 -> Driver -> Roll Back Driver to 3.5.2019.1.

8. Software (laptop / Jetson) explained

File	What it does
Navya.bat / navya.sh	menu on the laptop / on the Jetson: pre-flight, drive modes, viewers, calibration, tests
Navya Jetson.bat	laptop: one double-click finds the Jetson, opens its menu and dashboard
autopilot/config.py	every setting (Appendix B)
autopilot/navya_autopilot.py	the driving program (--phone --vision --steer --lidar --record --dry-run --show --replay)
autopilot/preflight.py	checks everything before a run; never moves the car
autopilot/navya_link.py	serial link to the Arduino (D, M, R, K commands; reads S lines)
autopilot/obstacle.py	depth image -> nearest obstacle in the car's path
autopilot/speed_planner.py	distance -> speed (CLEAR / SLOWING / STOPPED / RESUMING)
autopilot/battery.py	LiPo warning and stop
autopilot/lane.py, lane_follow.py	lane from the walls; pure-pursuit steering
autopilot/steering_calibrate.py	teaches full left / straight / full right (needs the pot)
autopilot/vision.py, vision_rules.py, vision_view.py	YOLO11n detection, people and stop-sign rules, viewer
autopilot/export_models.py	fast model copies: TensorRT (Jetson), OpenVINO, ONNX
autopilot/lidar.py, lidar_view.py	LiDAR reader and radar view
autopilot/corridor_view.py	lane, bird's-eye map, 3D map (never drives)
autopilot/phone_stream.py, live_settings.py	dashboard in the browser, live sliders
autopilot/ask_navya.py	Ask Navya: free Gemini, Groq fallback (display only)
autopilot/trip_recorder.py, trip_report.py	record videos + telemetry; trip summary
autopilot/calibrate_camera.py	measures camera height and tilt
autopilot/throttle_test.py	bench throttle test and PWM sweep
autopilot/test_logic.py	offline tests, no hardware (27 tests)
jetson/setup_jetson.sh, build_ch341.sh, find_jetson.py	Jetson setup, Arduino driver, finding the Jetson on the Wi-Fi
docs/wiring/make_wiring_pdf.py, make_wiring_image.py	wiring PDF and 16:9 image
.github/workflows/ci.yml	GitHub runs the tests and compiles the firmware on every push

Common commands (inside the autopilot folder)

```
python preflight.py                          check everything first
python navya_autopilot.py --phone            drive, watch in the browser
python navya_autopilot.py --phone --vision --record   drive + detection + recording
python navya_autopilot.py --dry-run --show   camera only, motors never driven
python navya_autopilot.py --phone --lidar    drive with the LiDAR too
python calibrate_camera.py --write           camera height and tilt
python throttle_test.py --sweep              find PWM_START (wheels up)
python steering_calibrate.py                 needs the pot
python trip_report.py                        summary of the last trip
python test_logic.py                         offline tests
```

Steering test scripts used on 9 Nov (in the session scratchpad, copied to Appendix C)

```
python pulse1.py COM14 255 1.0     one 1-second pulse, relay 1 (D5)
python pulse1.py COM14 -255 1.0    one 1-second pulse, relay 2 (D6)
```

The script refuses to send unless the radio is in AUTO, sends the M command every 50 ms, and always ends with the motor off. Maximum 1.4 s per pulse.

Ask Navya keys (still pending)

Copy `autopilot/keys.env.example` to `autopilot/keys.env`, then add a free GEMINI_API_KEY (aistudio.google.com) and GROQ_API_KEY (console.groq.com). Never paste keys in chat. No Anthropic key.

Jetson facts

Item	Value
Model / OS	Jetson Orin Nano Super, JetPack 6.2, L4T 36.4.7, MAXN_SUPER
Python	own environment ~/navya_autonomouscar/.venv: torch 2.8.0 (cu126), numpy < 2, opencv < 4.12
Detection	TensorRT engine, 23 ms per image
Recordings	/mnt/t7ssd/navya_trips (Samsung T7)
Network	laptop and Jetson on the same hotspot; ssh jetson; dashboard http://<jetson-ip>:8080
sudo	the user types the Jetson password in a terminal; never typed by Claude

9. Every problem found and how it was fixed

Problem	Cause	Fix / status
Car stopped too close (Demo 1)	31 kg coasts ~0.9 m after the brake	stop 2.0 m, slow-down 3.8 m, cruise 130
Slow to move off	long resume delay; car needs a push	resume 0.5 s, start boost 0.4 s
False obstacle at 1.8 m	floor points just above the clearance line	clearance 0.15 m, min 800 points
Tape lane failed	shiny floor reflections	lane from the walls (depth)
Arduino upload fails "cannot set com-state"	CH340 driver 3.9 on the clone	roll back to 3.5.2019.1 (happened again 7 Nov and 9 Nov on new USB ports)
~15 damaged serial lines a minute	115200 baud is 2% off on 16 MHz	250000 baud: 0 damaged in 1,200 lines
Brake flickered with stick untouched	stick centre drifted to ~1410 us	stop line 1300 us
Camera at USB 2 / stuck	shared cheap hub	camera straight into a port; auto reset
LiDAR saw the camera	same height	LiDAR raised above the camera
Floor became an obstacle after hard stops	taped camera mount nodded 8 deg	rigid mount, recalibrated
Laptop slept mid-test	hub could not charge it; 4% battery	charge directly; pre-flight fails below 20%
No Arduino port on the Jetson	JetPack kernel lacks CH341 driver	jetson/build_ch341.sh
Same line repeated on the Jetson	byte-wise readline with that driver	read chunks, split lines ourselves
Camera would not start on the Jetson	USB-C dock	plain cable into a Jetson port
GPU PyTorch broken	wrong torch build, numpy 2 pulled in	own venv, pinned versions
Laptop cannot reach the Jetson	college Wi-Fi isolates devices	hotspot
L298N silent (4 Nov)	enable jumper not bridging	ENB driven from A3
Radio NO_RC, channels 0	loose jumper wires at the Arduino	pushed in; rewired without the breadboard
L298N too weak going left on the ground	voltage drop in the L298N	not fixable; moved to BTS7960, then relays
BTS7960 does not move the motor	came unsoldered; 5 V solder bridge on L_EN	**board presumed dead**
Fuse soldered to wires; red B-wire; bare ends close		use the inline holder, black for negative, tape bare ends
iMAX B6AC showed 2.0V(1P)	wrong cell setting	12.0V(6P), 0.8 A
Relay clicks, motor does not move (9 Nov)	12 V path through the contacts incomplete	**OPEN - section 10**
LiDAR "lost" on the laptop	unknown (not motors, not load)	does not happen on the Jetson

10. Where we are stuck now, and how to get unstuck

Symptom: front lifted, Arduino sends a 1-second pulse to relay 1 (D5). The relay CLICKS (its LED lights), but the steering motor does not move.
Meaning: the Arduino, the firmware, the radio (AUTO) and the relay coil all work. The problem is on the 12 V side: the current is not getting from the battery, through the relay contacts, to the motor and back.

Relay 1 ON: the current must pass every link below. ONE loose link = motor does not move.



Relay 2 ON uses the mirror path: + -> fuse -> Relay 2 NO -> COM -> brown -> motor -> dark blue -> Relay 1 COM -> NC -> battery -

Most likely causes, in order

#	Possible cause	How to check (no multimeter needed)
1	Battery + was not connected during the test (earlier instructions said keep it off while wiring)	make sure the fuse lead is on battery + before "Ready"
2	A wire is on the wrong screw (COM is the MIDDLE screw of each relay)	read the small print next to each screw: NO, COM, NC
3	A screw is clamped on the wire's plastic, not on bare copper	pull each wire gently; strip 6-7 mm and re-tighten
4	Relay 2's NC is not on battery - (the return path for relay 1)	follow the black wire from relay 2 NC to battery -
5	The NO-NO link or NC-NC link is missing	both NO joined, both NC joined
6	Fuse not seated in the holder or blown	swap in a spare 10 A fuse
7	Motor wire came off	dark blue on relay 1 COM, brown on relay 2 COM

Quick test without the Arduino (battery + connected, front lifted)

1. Disconnect the dark blue wire from relay 1 COM and touch it for one second to the fuse lead (+12 V), with the brown wire still on relay 2 COM. If the wheels turn: relay 2's NC and the battery - path are fine, and the fault is at relay 1 (NO or COM screw).
2. If nothing turns: the fault is on the battery - side (relay 2 NC, the NC link, or the black wire).

After it moves

1. Pulse the other direction (relay 2, D6).
2. Ground test: centre -> full left, the move the L298N could not do.
3. Then open-loop steering (find the ends by stall, centre by time), and later the pot.
4. Commit the relay firmware with the co-author line, and update the README and wiring docs.

11. What is left (full to-do list)

#	Task	Waiting for
1	Fix the relay 12 V path and get the steering motor moving (section 10)	terminal photo / check
2	Test both directions lifted, then centre -> full left on the ground	item 1
3	Open-loop steering code: ends by stall, centre by time, camera correction	item 2
4	Commit and push the relay firmware; update README steering section and wiring docs	item 2 + the user's OK
5	Fit the potentiometer (option A or B), then run steering_calibrate.py	time for mechanical work
6	Fit the 2 limit switches (A1/A2)	mounting
7	Lane following run in the corridor at creep speed with --record	items 3/5
8	Brake wires to D11/D12. The earlier plan put them through the relay module, which is now used for steering: needs a second relay module, or a multimeter check of the brake wire voltage first	decision + multimeter
9	Buy a multimeter	purchase
10	Replacement BTS7960 (buy pre-soldered) if relays are not good enough	decision
11	One battery for both controllers (the second LiPo as a swap)	decision
12	12 V step-down converter (LiPo -> 5 A fuse -> 12 V >= 5 A -> Jetson), then drive with the Jetson	converter (ordered)
13	Person test on the ground: car must drive on after the person steps aside	test time
14	--lidar ground run (with the Jetson)	item 12
15	Battery monitor: 2x 47k + 2x 10k resistors on A4/A5	4 resistors
16	Ask Navya keys in autopilot/keys.env, then a live test	the user's free keys
17	Change the Jetson password with `passwd` (it was written in chat once)	the user
18	MIT licence for the repository (offered, no answer yet)	the user's choice
19	Later: Arduino Mega for wheel-speed sensors; LiDAR mapping	budget

12. Parts, bill and what we own

Robu.in order (31 Oct, Rs 2,302 incl. GST), arrived 7 Nov

Qty	Part	SKU	Status
1	SmartElex BTS7960 motor driver (43 A, 6-27 V)	R189064	**presumed dead**
2	10 kOhm potentiometer, 15 mm shaft	607345	**not fitted**
2	KW4 roller-lever micro switch	R132437	**not fitted**
1	Inline blade fuse holders (5 pcs)	699636	**in use**
4	10 A blade fuse (Littelfuse ATOF 0287010, 32 V)	R182177	**in use, proven good**
2 m	18 AWG silicone wire, red	R164399	**in use**
2 m	18 AWG silicone wire, black	R164398	**in use**

Considered but not bought (kept in docs/navya_bill_robu.xlsx, marked No): 180 kg-cm servo BL51180-24V and its linkage (professor: keep the toy steering), e-stop button and relay, battery isolator switches, XT60 pigtails, 256 GB microSD.

Grey battery charging (iMAX B6AC)

Setting	Value
Program	Pb (lead-acid) CHARGE
Voltage	12.0V(6P)
Current	0.8 A (0.1 x 8 Ah; maximum 2.4 A)
Label limits	cyclic charge 14.5-14.9 V, max 2.4 A
Result 5 Nov	finished at 14.40 V, 0 mAh: it was already full

13. Rules we work by

- The car never moves without the user's go-ahead: wheels up (or front lifted), everyone clear, and the user says "Ready".
- Keep battery + disconnected while wiring.
- Steering: keep the toy's own motor and gearbox (professor).
- Never type the user's passwords, OTPs or logins; the user types sudo passwords themselves.
- No Anthropic API key in the project; free Gemini + Groq with fallback; keys never pasted in chat.
- No simulation work; scripted offline tests are fine.
- System and driver settings (like the CH340 rollback) are changed by the user.
- Commit and push only with the user's OK. Commit messages end with "Co-Authored-By: Claude Opus 5.5 <noreply@anthropic.com>".
- One step at a time, simple words, tables.

Appendix A: relay firmware change (exact diff, not committed yet)

```
diff --git a/firmware/navya_drive/navya_drive.ino b/firmware/navya_drive/navya_drive.ino
index 70ef38b..24948a0 100644
--- a/firmware/navya_drive/navya_drive.ino
+++ b/firmware/navya_drive/navya_drive.ino
@@ -3,8 +3,8 @@

   Rear: two BLDC hub motors through their YL-12 controllers (throttle PWM stands
   in for the hand throttle). Front: the toy's own steering motor through a
-  BTS7960 driver, with a potentiometer on the steering shaft for the angle and
-  two limit switches at full left / full right.
+  2-channel relay module (STEER_RELAYS) or a BTS7960 driver, with a potentiometer
+  on the steering shaft for the angle and two limit switches at full left / full right.

   Modes, picked by the FlySky CH5 switch:
     MANUAL  - right stick up/down (CH2) = forward / brake,
@@ -84,6 +84,15 @@ const uint16_t STEER_FAULT_HOLD_MS = 2000;
 const uint16_t POT_MIN_VALID = 15;     // readings outside this = broken pot wire
 const uint16_t POT_MAX_VALID = 1008;
 const uint8_t RAW_STEER_LIMIT = 255;
+// Steering driver. true = a 2-channel relay module (trigger jumpers on LOW) wired as
+// an H-bridge: relay 1 on D5, relay 2 on D6; relay 1 COM to the motor's dark blue,
+// relay 2 COM to its brown, both NO to +12 V through the fuse, both NC to battery -.
+// Both off or both on = motor stopped, so it cannot short the battery. Full power or
+// off, no PWM. false = a BTS7960 / L298N driver with PWM on D5 / D6 and enable on A3.
+const bool STEER_RELAYS = true;
+const uint8_t STEER_RELAY_ON_PWM = 40;   // |pwm| from this up switches a relay on
+const uint16_t STEER_RELAY_MIN_MS = 60;  // least time a relay stays on (no chatter)
+const uint16_t STEER_RELAY_GAP_MS = 100; // both off at least this long before one switches on

 // ---------------- Radio ----------------
 // Our FS-CT6B: CH2 centre ~1410-1464 (it drifts with the trim), full up ~1756 with SWA down
@@ -304,9 +313,35 @@ uint16_t batteryMv(uint8_t pin) {
 bool limitLeft() { return digitalRead(PIN_LIMIT_LEFT) == LOW; }
 bool limitRight() { return digitalRead(PIN_LIMIT_RIGHT) == LOW; }

+// Relay steering (STEER_RELAYS). Relay inputs work like the brake wires: drive LOW =
+// relay on, float = off, so a resetting or unpowered Arduino leaves both relays off.
+int8_t relayDir = 0;          // +1 = relay 1 (D5) on, -1 = relay 2 (D6) on, 0 = both off
+uint32_t relayChangeMs = 0;
+
+// force = switch now (used for stops); otherwise changes are spaced out so the relays
+// never chatter, and a reversal always passes through "both off" for STEER_RELAY_GAP_MS.
+void steerRelays(int8_t dir, uint32_t nowMs, bool force) {
+  if (!force) {
+    if (relayDir != 0 && dir != 0 && dir != relayDir) dir = 0;   // reverse via off
+    if (dir == relayDir) return;
+    uint16_t wait = relayDir == 0 ? STEER_RELAY_GAP_MS : STEER_RELAY_MIN_MS;
+    if (nowMs - relayChangeMs < wait) return;
+  } else if (dir == relayDir) {
+    return;
+  }
+  setActiveLow(PIN_STEER_RPWM, dir > 0);
+  setActiveLow(PIN_STEER_LPWM, dir < 0);
+  relayDir = dir;
+  relayChangeMs = nowMs;
+}
+
 void steerOff() {
-  analogWrite(PIN_STEER_RPWM, 0);
-  analogWrite(PIN_STEER_LPWM, 0);
+  if (STEER_RELAYS) {
+    steerRelays(0, millis(), true);
+  } else {
+    analogWrite(PIN_STEER_RPWM, 0);
+    analogWrite(PIN_STEER_LPWM, 0);
+  }
   digitalWrite(PIN_STEER_ENABLE, LOW);
   // The stall timer only runs while the motor is driven.
   stallRefPot = potRaw;
@@ -331,8 +366,13 @@ void steerMotorRaw(int16_t pwm, uint32_t nowMs) {
     return;
   }
   digitalWrite(PIN_STEER_ENABLE, HIGH);
-  analogWrite(PIN_STEER_RPWM, pwm > 0 ? pwm : 0);
-  analogWrite(PIN_STEER_LPWM, pwm < 0 ? -pwm : 0);
+  if (STEER_RELAYS) {
+    int8_t dir = pwm >= STEER_RELAY_ON_PWM ? 1 : (pwm <= -STEER_RELAY_ON_PWM ? -1 : 0);
+    steerRelays(dir, nowMs, dir == 0);
+  } else {
+    analogWrite(PIN_STEER_RPWM, pwm > 0 ? pwm : 0);
+    analogWrite(PIN_STEER_LPWM, pwm < 0 ? -pwm : 0);
+  }
 }

 // pwm > 0 turns the wheels right. Limit switches block further travel.
@@ -410,6 +450,7 @@ void printConfig() {
   Serial.print(F(",require_rc=")); Serial.print(REQUIRE_RC);
   Serial.print(F(",host_timeout_ms=")); Serial.print(HOST_TIMEOUT_MS);
```

```
   Serial.print(F(",steering=")); Serial.print(STEERING_INSTALLED);
+  Serial.print(F(",steer_relays=")); Serial.print(STEER_RELAYS);
   Serial.print(F(",cal_ok=")); Serial.print(calOk);
   Serial.print(F(",cal_left=")); Serial.print(cal.left);
   Serial.print(F(",cal_center=")); Serial.print(cal.center);
@@ -513,8 +554,10 @@ void setup() {
   setActiveLow(PIN_DIR_RIGHT, INVERT_RIGHT);
   pinMode(PIN_LED, OUTPUT);

-  pinMode(PIN_STEER_RPWM, OUTPUT);
-  pinMode(PIN_STEER_LPWM, OUTPUT);
+  if (!STEER_RELAYS) {         // relay inputs stay floating (= off) until used
+    pinMode(PIN_STEER_RPWM, OUTPUT);
+    pinMode(PIN_STEER_LPWM, OUTPUT);
+  }
   pinMode(PIN_STEER_ENABLE, OUTPUT);
   steerOff();
   pinMode(PIN_LIMIT_LEFT, INPUT_PULLUP);
```

Appendix B: config.py (all settings)

autopilot/config.py

C:\Users\connt\dev\navya_autonomouscar\autopilot\config.py - 145 lines

```
"""Navya autopilot settings. Every laptop script reads this file.

Distances are metres. Speeds are 0..1000, a share of the Arduino's
PWM_START..PWM_MAX throttle range (see firmware/navya_drive/navya_drive.ino).
"""

# --- Arduino link ---
SERIAL_PORT = None          # e.g. "COM5"; None = auto-detect the Uno
SERIAL_BAUD = 250000          # must match Serial.begin() in navya_drive.ino

# --- RealSense depth stream ---
DEPTH_WIDTH = 640
DEPTH_HEIGHT = 480
DEPTH_FPS = 30
DEPTH_STRIDE = 2            # use every 2nd pixel each way; plenty for this job

# --- Camera mounting (python calibrate_camera.py --write fills the first two) ---
CAMERA_HEIGHT_M = 0.554      # lens centre above the floor
CAMERA_PITCH_DEG = 4.4      # positive = tilted down
CAMERA_LATERAL_OFFSET_M = 0.0   # positive = camera right of the car's centre line
CAMERA_SETBACK_M = 0.0      # lens behind the front bumper; subtracted from distances

# --- Driving corridor: only points inside this box count as obstacles ---
CORRIDOR_HALF_WIDTH_M = 0.45    # car is ~0.62 m wide; half-width plus a 0.14 m margin
FLOOR_CLEARANCE_M = 0.15       # ignore anything lower (the floor itself, tape, cables)
MAX_OBSTACLE_HEIGHT_M = 1.6     # ignore anything higher (ceiling, overhead lights)
DETECT_MIN_M = 0.2
DETECT_MAX_M = 4.5
OBSTACLE_PERCENTILE = 5         # nearest 5% of corridor points sets the distance
MIN_OBSTACLE_POINTS = 800      # fewer points than this = sensor noise, not an obstacle

# The D435i cannot see closer than ~0.3 m: those pixels read 0. If most of the
# centre of the image has no depth, something is right in front of the lens.
# Shiny floors can also read as "no depth"; set 1.01 to switch this check off.
BLIND_CHECK_FRACTION = 0.6

# --- Speed plan ---
CRUISE_SPEED = 130
CREEP_SPEED = 80            # slowest moving speed while closing in on an obstacle
SLOW_DOWN_DIST_M = 3.8      # start slowing down when an obstacle is this close
STOP_DIST_M = 2.0           # stop here; the car coasts ~0.9 m more (first ground run)
RESUME_DIST_M = 2.4         # path counts as clear again beyond this distance...
RESUME_DELAY_S = 0.5        # ...for this long, then the car moves off again
START_BOOST_SPEED = 250     # extra throttle for moving off from a stop (31 kg needs a push)...
START_BOOST_S = 0.4         # ...for this long, then back to the planned speed
DISTANCE_HISTORY = 3        # use the nearest reading of the last N frames

# --- Straight-line trim: positive = right wheel faster. Car veers right -> raise it.
TRIM = 0.0

# --- Battery monitor (needs the 47k/10k dividers on A4/A5; 6S LiPo) ---
BATTERY_WARN_V = 22.2           # below this (3.7 V/cell): warning on phone and terminal
BATTERY_STOP_V = 21.6           # below this for BATTERY_STOP_S while driving: stop and stay stopped
BATTERY_STOP_S = 3.0            # (voltage dips briefly under load; ignore short dips)

# --- Steering and lane following (python navya_autopilot.py --steer) ---
# Steering commands run -1000 (full left) .. 1000 (full right); steering_calibrate.py
# teaches the Arduino where full left / straight / full right are.
WHEELBASE_M = 0.745             # front to rear axle (measured)
CAMERA_TO_REAR_AXLE_M = 0.95    # camera lens is this far in front of the rear axle (measure)
STEER_MAX_DEG = 25.0            # front-wheel angle at full lock (measure after calibrating)
LOOKAHEAD_M = 1.5               # aim at the lane centre this far ahead (longer = smoother, lazier)
STEER_RATE_PER_S = 1500         # steering command may change at most this much per second
LANE_LOST_HOLD_S = 0.5          # lane missing for less than this: keep the last steering
LANE_LOST_SPEED = 80            # lane missing for longer: steer straight and creep at this speed

# --- LiDAR (LDROBOT D500 kit on the laptop's USB; python lidar_view.py / --lidar) ---
LIDAR_PORT = None               # e.g. "COM7"; None = find it automatically
LIDAR_BAUD = 230400
LIDAR_ANGLE_OFFSET_DEG = -1.0   # sensor angle that points straight ahead (measured 2 Nov 2026)
LIDAR_CLOCKWISE = True          # sensor angles grow clockwise seen from above (checked on the car)
LIDAR_SETBACK_M = 0.365         # LiDAR behind the front bumper; subtracted from distances
LIDAR_LATERAL_OFFSET_M = -0.10  # positive = LiDAR right of the car's centre line (here: 10 cm left)
LIDAR_IGNORE_WITHIN_M = 0.15    # ignore returns this close (the car's own mast, cables)
LIDAR_MIN_INTENSITY = 10        # weaker returns are noise (0..255)
LIDAR_MIN_POINTS = 3            # points needed in the corridor to count as an obstacle
LIDAR_MAX_AGE_S = 0.5           # with --lidar: no new scan for this long = stop, as for lost depth

# --- Ask Navya (phone button; display only) ---
ASK_PROVIDERS = ["gemini", "groq"]     # tried in this order; the next one answers if one fails
GEMINI_MODEL = "gemini-3.5-flash-lite" # free tier, accepts images
GROQ_MODEL = "qwen/qwen3.8-27b"        # free tier, accepts images
ASK_TIMEOUT_S = 20

# --- Object detection (python vision_view.py) ---
DETECT_CONFIDENCE = 0.4     # ignore detections the model is less sure about than this
```

```
DETECT_IMAGE_SIZE = 480     # model input size; smaller = faster, larger = sees smaller things
                            # (after changing it, run export_models.py again)
DETECT_BACKEND = "auto"     # auto = fastest available: "tensorrt" (Jetson), "openvino" (Intel GPU), "onnx", "torch"
DETECT_CLASSES = [          # what to look for ([] = all 80 COCO classes)
    "person", "bicycle", "car", "motorcycle", "bus", "truck", "stop sign", "dog", "cat",
    "chair", "bench", "backpack", "handbag", "suitcase", "bottle", "cup", "potted plant",
    "dining table", "couch", "tv", "laptop", "cell phone",
]
DETECT_THREADS = 4          # CPU threads for YOLO; the rest stay free for the driving loop

# --- Object-aware driving (python navya_autopilot.py --vision) ---
# Detection only ever makes the car more careful; the depth obstacle check always runs.
PERSON_EXTRA_MARGIN_M = 1.0     # people: stop this much further away than for other obstacles
PERSON_SIDE_MARGIN_M = 0.3      # a person this far outside the car's corridor still counts
STOP_SIGN_TRIGGER_M = 2.5       # stop when a stop sign is closer than this...
STOP_SIGN_WAIT_S = 3.0          # ...wait this long...
STOP_SIGN_IGNORE_S = 5.0        # ...then drive on, ignoring stop signs for this long to pass it
VISION_MAX_AGE_S = 0.5          # detections older than this are ignored

# --- Trip recorder (python navya_autopilot.py --record) ---
RECORD_DIR = "auto"         # auto = the SSD below if mounted, else an SD card / USB stick,
                            # else autopilot/recordings; or a folder path
RECORD_SSD_DIR = "/mnt/t7ssd/navya_trips"   # Jetson: Samsung T7 (shared with the drone project)
RECORD_FPS = 15             # video frames per second
RECORD_LOG_HZ = 10          # telemetry rows per second
RECORD_MIN_FREE_GB = 2.0    # stop recording (not driving) when the disk gets this full

# --- Corridor viewer and lane finder (python corridor_view.py) ---
# "walls":  lane = free corridor between the walls, from depth (works on shiny floors)
# "strips": lane = floor between two black tape/tile strips, from the colour camera
LANE_SOURCE = "walls"
WALL_MIN_HEIGHT_M = 0.15    # wall points: from this height...
WALL_MAX_HEIGHT_M = 1.5     # ...up to this one
WALL_MIN_FORWARD_M = 0.5
WALL_MAX_FORWARD_M = 5.0
WALL_MIN_LATERAL_M = 0.35   # ignore things right in front of the car (people, boxes) as walls
WALL_FIT_TOLERANCE_M = 0.10
WALL_MAX_LANE_WIDTH_M = 6.0

# Both lane sources: sanity checks and drawing range.
LANE_MIN_SAMPLES = 8        # 10 cm steps (walls) / 4 cm rows (strips) needed per edge
LANE_MIN_WIDTH_M_TOTAL = 0.6    # a lane narrower than this is rejected
LANE_MAX_STRIP_ANGLE_DEG = 10   # the two edges must be roughly parallel
LANE_MAX_HEADING_DEG = 35
LANE_MAX_FORWARD_M = 4.0    # edges are fitted and drawn up to here

# Strip lane only (LANE_SOURCE = "strips").
LANE_DARK_MAX = 70          # a floor cell is black tile if darker than this (0-255)...
LANE_DARK_RATIO = 0.5       # ...and darker than this share of the floor's median brightness
LANE_MIN_WIDTH_M = 0.05     # ignore dark bits narrower than this (grout lines, cables)
LANE_MAX_WIDTH_M = 3.5

# Viewer display.
VIEW_FPS = 15               # depth + colour together; 15 is safe even on USB 2
VIEW_POINT_STRIDE = 3       # 3D view draws every 3rd pixel each way
BEV_RES_M = 0.02            # bird's-eye floor map: 2 cm per cell
BEV_HALF_WIDTH_M = 2.0      # map covers 2 m either side of the car...
BEV_MIN_FORWARD_M = 0.5     # ...from 0.5 m...
BEV_MAX_FORWARD_M = 6.0     # ...to 6 m ahead
```

Appendix C: steering test script

pulse1.py

C:\Users\connt\dev\navya_autonomouscar\docs\report\pulse1.py - 29 lines

```
"""One steering pulse in one direction. Usage: pulse1.py PORT PWM SECONDS (PWM may be negative)."""
import sys, time
sys.path.insert(0, r"C:\Users\connt\dev\navya_autonomouscar\autopilot")
from navya_link import NavyaLink

port, pwm, secs = sys.argv[1], int(sys.argv[2]), float(sys.argv[3])
assert abs(pwm) <= 255 and secs <= 1.4     # firmware stall cut-off is 1.5 s
link = NavyaLink(port)
try:
    assert link.wait_ready(), "no reply from Arduino"
    time.sleep(0.5)
    s = link.status
    print("start:", s.mode, "state", s.steer_state)
    assert s.mode == "AUTO", "not in AUTO, nothing sent"
    end = time.monotonic() + secs
    states = set()
    while time.monotonic() < end:
        link.steer_raw(pwm)
        time.sleep(0.05)
        s = link.status
        states.add(s.steer_state)
        if s.mode != "AUTO":
            print("left AUTO, stopping"); break
    print(f"pulse {pwm:+d} for {secs}s: states {sorted(states)}")
finally:
    link.steer_raw(0)
    time.sleep(0.2)
    link.close()
print("done, motor off")
```

Appendix D: full Arduino firmware

firmware/navya_drive/navya_drive.ino

C:\Users\connt\dev\navya_autonomouscar\firmware\navya_drive\navya_drive.ino - 656 lines

```
/*
  Navya drive firmware v2 - Arduino Uno

  Rear: two BLDC hub motors through their YL-12 controllers (throttle PWM stands
  in for the hand throttle). Front: the toy's own steering motor through a
  2-channel relay module (STEER_RELAYS) or a BTS7960 driver, with a potentiometer
  on the steering shaft for the angle and two limit switches at full left / full right.

  Modes, picked by the FlySky CH5 switch:
    MANUAL  - right stick up/down (CH2) = forward / brake,
              right stick left/right (CH1) = steering.
    AUTO    - the laptop drives and steers over USB serial.
              Pulling CH2 back still stops the car (operator override).
    NO_RC   - no receiver signal: motors off, brakes on, steering motor off.

  The car also stops if the laptop goes quiet for HOST_TIMEOUT_MS (steering
  then holds straight).

  Serial protocol, 250000 baud (exact on a 16 MHz Uno; 115200 is 2% off and a CH340
  clone then drops bytes), one command per line:
    D,<left>,<right>,<brake>[,<steer>]  speed 0..1000 per wheel, brake 0/1,
                                        steer -1000 (full left)..1000 (full right)
    R,<pwmLeft>,<pwmRight>              raw throttle PWM, for calibration only
    M,<pwm>                             raw steering motor -255..255 (+ = RPWM),
                                        for steering calibration only (AUTO)
    K,L / K,C / K,R                     store the current pot reading as full left /
                                        straight / full right (kept in EEPROM)
    K,I,<0|1>                           1 = RPWM turns the wheels LEFT
    K,X                                 forget the steering calibration
    ?                                   print the settings below
  Every STATUS_PERIOD_MS the Uno replies:
    S,<mode>,<ch2_us>,<ch5_us>,<host_ok>,<pwmLeft>,<pwmRight>,<brake>,
      <ch1_us>,<steer_target>,<steer_pos>,<pot_raw>,<limit_left>,<limit_right>,<steer_state>,
      <battery_left_mV>,<battery_right_mV>       (0 = no battery divider connected)
*/

#include <EEPROM.h>

// ---------------- Pins ----------------
const uint8_t PIN_RC_MODE = 2;         // receiver CH5 signal (interrupt pin)
const uint8_t PIN_RC_THROTTLE = 3;     // receiver CH2 signal (interrupt pin)
const uint8_t PIN_RC_STEER = 4;        // receiver CH1 signal (pin-change interrupt)
const uint8_t PIN_STEER_RPWM = 5;      // BTS7960 RPWM
const uint8_t PIN_STEER_LPWM = 6;      // BTS7960 LPWM
const uint8_t PIN_DIR_RIGHT = 7;       // controller F/R silver wire
const uint8_t PIN_DIR_LEFT = 8;
const uint8_t PIN_THROTTLE_LEFT = 9;   // controller throttle purple wire
const uint8_t PIN_THROTTLE_RIGHT = 10;
const uint8_t PIN_BRAKE_LEFT = 11;     // controller brake wire (optional, active low)
const uint8_t PIN_BRAKE_RIGHT = 12;
const uint8_t PIN_LED = 13;
const uint8_t PIN_STEER_POT = A0;      // potentiometer wiper (ends to 5V and GND)
const uint8_t PIN_LIMIT_LEFT = A1;     // limit switch to GND, closed = at full left
const uint8_t PIN_LIMIT_RIGHT = A2;    // limit switch to GND, closed = at full right
const uint8_t PIN_STEER_ENABLE = A3;   // BTS7960 R_EN + L_EN
const uint8_t PIN_BATT_LEFT = A4;      // left LiPo through a 47k / 10k divider
const uint8_t PIN_BATT_RIGHT = A5;     // right LiPo through a 47k / 10k divider

// ---------------- Battery monitor ----------------
// LiPo + -> 47k -> pin -> 10k -> GND (battery negatives joined to Arduino GND).
// 25.2 V full charge reads as 4.42 V at the pin.
const float BATT_DIVIDER = (47.0 + 10.0) / 10.0;
const float BATT_ADC_REF_V = 5.0;      // the Uno's 5 V; raise/lower to match a multimeter

// ---------------- Throttle calibration ----------------
// Find PWM_START with: python throttle_test.py --sweep (wheels off the ground)
const uint8_t PWM_IDLE = 0;            // throttle output when stopped
const uint8_t PWM_START = 75;          // lowest PWM at which the wheels turn (bench: ~75-78)
const uint8_t PWM_MAX = 130;           // PWM sent for speed 1000 (this is the speed cap)
const uint8_t PWM_RAW_LIMIT = 160;     // ceiling for R, calibration commands
const bool INVERT_LEFT = false;        // true = drive this wheel's F/R wire low
const bool INVERT_RIGHT = false;
const uint16_t RAMP_UP_PER_10MS = 5;   // speed units gained per 10 ms (0 -> 1000 in 2 s)

// ---------------- Steering ----------------
const bool STEERING_INSTALLED = true;  // false = never drive the steering motor
const uint8_t STEER_PWM_MAX = 200;     // full effort far from the target
const uint8_t STEER_PWM_MIN = 70;      // least effort that still moves the toy gearbox
const int16_t STEER_DEADBAND = 25;     // within 2.5% of travel: close enough, motor off
const int16_t STEER_SLOW_ZONE = 250;   // closer than this, effort drops towards STEER_PWM_MIN
const int16_t STEER_TARGET_LIMIT = 900; // never aim past 90% of the calibrated travel
const uint16_t STEER_STALL_MS = 1500;  // driving this long without the pot moving = stuck
const uint16_t STEER_FAULT_HOLD_MS = 2000;
const uint16_t POT_MIN_VALID = 15;     // readings outside this = broken pot wire
const uint16_t POT_MAX_VALID = 1008;
const uint8_t RAW_STEER_LIMIT = 255;
```

```
// Steering driver. true = a 2-channel relay module (trigger jumpers on LOW) wired as
// an H-bridge: relay 1 on D5, relay 2 on D6; relay 1 COM to the motor's dark blue,
// relay 2 COM to its brown, both NO to +12 V through the fuse, both NC to battery -.
// Both off or both on = motor stopped, so it cannot short the battery. Full power or
// off, no PWM. false = a BTS7960 / L298N driver with PWM on D5 / D6 and enable on A3.
const bool STEER_RELAYS = true;
const uint8_t STEER_RELAY_ON_PWM = 40;   // |pwm| from this up switches a relay on
const uint16_t STEER_RELAY_MIN_MS = 60;  // least time a relay stays on (no chatter)
const uint16_t STEER_RELAY_GAP_MS = 100; // both off at least this long before one switches on

// ---------------- Radio ----------------
// Our FS-CT6B: CH2 centre ~1410-1464 (it drifts with the trim), full up ~1756 with SWA down
// (SWA up = dual rate, ~1644), full back ~1120-1250.
const bool REQUIRE_RC = true;          // false = bench testing without the transmitter
const uint16_t RC_AUTO_ABOVE_US = 1700;
const uint16_t RC_STOP_BELOW_US = 1300;  // well below the centre, so a centred stick never brakes
const uint16_t RC_FORWARD_FROM_US = 1520;
const uint16_t RC_FORWARD_FULL_US = 1740;
const uint16_t RC_STEER_CENTER_US = 1500;   // CH1 stick centred (check in the S, line)
const uint16_t RC_STEER_SPAN_US = 250;      // stick this far from centre = full lock
const uint16_t RC_STEER_DEADBAND_US = 25;
const uint16_t RC_TIMEOUT_MS = 100;

// ---------------- Laptop link ----------------
const uint16_t HOST_TIMEOUT_MS = 300;
const uint16_t STATUS_PERIOD_MS = 100;

enum Mode { MODE_NO_RC, MODE_MANUAL, MODE_AUTO };

struct RcSnapshot {
  uint16_t throttleUs;
  uint16_t modeUs;
  uint16_t steerUs;
  bool fresh;        // CH2 and CH5 both arriving
  bool steerFresh;   // CH1 arriving
};

struct SteerCal {
  uint16_t magic;
  uint16_t left;
  uint16_t center;
  uint16_t right;
  uint8_t invert;    // 1 = RPWM turns the wheels left
};
const uint16_t CAL_MAGIC = 0x4E56;  // "NV"
const int CAL_ADDR = 0;

// Written by the pin interrupts.
volatile uint32_t throttleRiseUs = 0;
volatile uint32_t modeRiseUs = 0;
volatile uint32_t steerRiseUs = 0;
volatile uint16_t throttlePulseUs = 0;
volatile uint16_t modePulseUs = 0;
volatile uint16_t steerPulseUs = 0;
volatile uint32_t throttleSeenMs = 0;
volatile uint32_t modeSeenMs = 0;
volatile uint32_t steerSeenMs = 0;

// Last laptop command.
int16_t hostLeft = 0;
int16_t hostRight = 0;
bool hostBrake = true;
bool hostRaw = false;
uint8_t rawLeft = 0;
uint8_t rawRight = 0;
int16_t hostSteer = 0;
bool hostSteerRaw = false;
int16_t rawSteer = 0;
uint32_t hostCmdMs = 0;
bool hostSeen = false;

// Ramped speeds actually being sent, 0..1000.
int16_t currentLeft = 0;
int16_t currentRight = 0;
uint32_t lastRampMs = 0;

uint8_t outPwmLeft = 0;
uint8_t outPwmRight = 0;
bool outBrake = true;

// Steering state.
SteerCal cal;
bool calOk = false;
uint16_t potRaw = 0;
int16_t steerPos = 0;
int16_t steerTarget = 0;
const char *steerState = "OFF";
uint32_t steerFaultUntil = 0;
uint16_t stallRefPot = 0;
uint32_t stallRefMs = 0;
uint32_t lastSteerMs = 0;

char lineBuf[40];
uint8_t lineLen = 0;
uint32_t lastStatusMs = 0;

```

```
183 void isrThrottle() {
184   uint32_t now = micros();
185   if (digitalRead(PIN_RC_THROTTLE)) {
186     throttleRiseUs = now;
187   } else {
188     uint32_t width = now - throttleRiseUs;
189     if (width >= 800 && width <= 2200) {
190       throttlePulseUs = width;
191       throttleSeenMs = millis();
192     }
193   }
194 }
195
196 void isrMode() {
197   uint32_t now = micros();
198   if (digitalRead(PIN_RC_MODE)) {
199     modeRiseUs = now;
200   } else {
201     uint32_t width = now - modeRiseUs;
202     if (width >= 800 && width <= 2200) {
203       modePulseUs = width;
204       modeSeenMs = millis();
205     }
206   }
207 }
208
209 // CH1 on D4 (PD4): pin-change interrupt; only D4 is enabled in this group.
210 ISR(PCINT2_vect) {
211   uint32_t now = micros();
212   if (PIND & _BV(PIND4)) {
213     steerRiseUs = now;
214   } else {
215     uint32_t width = now - steerRiseUs;
216     if (width >= 800 && width <= 2200) {
217       steerPulseUs = width;
218       steerSeenMs = millis();
219     }
220   }
221 }
222
223 RcSnapshot readRc(uint32_t nowMs) {
224   noInterrupts();
225   uint16_t thr = throttlePulseUs;
226   uint16_t mode = modePulseUs;
227   uint16_t steer = steerPulseUs;
228   uint32_t thrMs = throttleSeenMs;
229   uint32_t modeMs = modeSeenMs;
230   uint32_t steerMs = steerSeenMs;
231   interrupts();
232
233   RcSnapshot rc;
234   rc.throttleUs = thr;
235   rc.modeUs = mode;
236   rc.steerUs = steer;
237   rc.fresh = thr != 0 && mode != 0 &&
238              nowMs - thrMs < RC_TIMEOUT_MS && nowMs - modeMs < RC_TIMEOUT_MS;
239   rc.steerFresh = steer != 0 && nowMs - steerMs < RC_TIMEOUT_MS;
240   return rc;
241 }
242
243 Mode decideMode(const RcSnapshot &rc) {
244   if (!rc.fresh) return REQUIRE_RC ? MODE_NO_RC : MODE_AUTO;
245   return rc.modeUs > RC_AUTO_ABOVE_US ? MODE_AUTO : MODE_MANUAL;
246 }
247
248 // Brake and F/R wires are pulled up inside the controller: drive LOW to
249 // activate, float (INPUT) to release.
250 void setActiveLow(uint8_t pin, bool active) {
251   if (active) {
252     digitalWrite(pin, LOW);
253     pinMode(pin, OUTPUT);
254   } else {
255     pinMode(pin, INPUT);
256   }
257 }
258
259 uint8_t speedToPwm(int16_t speed) {
260   if (speed <= 0) return PWM_IDLE;
261   if (speed > 1000) speed = 1000;
262   return PWM_START + (uint8_t)((uint32_t)(PWM_MAX - PWM_START) * speed / 1000);
263 }
264
265 // ---------------- Steering ----------------
266
267 void loadCal() {
268   EEPROM.get(CAL_ADDR, cal);
269   calOk = cal.magic == CAL_MAGIC && cal.left != cal.right &&
270           ((cal.left < cal.center && cal.center < cal.right) ||
271            (cal.right < cal.center && cal.center < cal.left));
272 }
273
274 void saveCal() {
275   cal.magic = CAL_MAGIC;
276   EEPROM.put(CAL_ADDR, cal);
277   loadCal();
278 }
```

```
279
280 uint16_t readPot() {
281   uint16_t sum = 0;
282   for (uint8_t i = 0; i < 4; i++) sum += analogRead(PIN_STEER_POT);
283   return sum / 4;
284 }
285
286 // Pot reading -> -1000 (full left) .. 0 (straight) .. 1000 (full right),
287 // whichever way round the potentiometer is fitted.
288 int16_t potToPos(uint16_t raw) {
289   long d = (long)raw - cal.center;
290   long toLeft = (long)cal.left - cal.center;
291   long toRight = (long)cal.right - cal.center;
292   long p = (d * toLeft > 0) ? -1000L * d / toLeft : 1000L * d / toRight;
293   return (int16_t)constrain(p, -1100L, 1100L);
294 }
295
296 // Battery voltage in millivolts (0 when nothing is connected to the pin).
297 // An unconnected pin floats and can read any voltage, so first test it with the
298 // internal pull-up: the divider's 10k to GND drags the pin well down, a bare pin
299 // reads near 5 V.
300 uint16_t batteryMv(uint8_t pin) {
301   pinMode(pin, INPUT_PULLUP);
302   analogRead(pin);                       // settle the ADC on this pin
303   bool unconnected = analogRead(pin) > 1000;
304   pinMode(pin, INPUT);
305   if (unconnected) return 0;
306   analogRead(pin);
307   uint16_t sum = 0;
308   for (uint8_t i = 0; i < 4; i++) sum += analogRead(pin);
309   float volts = sum / 4.0 * BATT_ADC_REF_V / 1023.0 * BATT_DIVIDER;
310   return volts < 3.0 ? 0 : (uint16_t)(volts * 1000.0);
311 }
312
313 bool limitLeft() { return digitalRead(PIN_LIMIT_LEFT) == LOW; }
314 bool limitRight() { return digitalRead(PIN_LIMIT_RIGHT) == LOW; }
315
316 // Relay steering (STEER_RELAYS). Relay inputs work like the brake wires: drive LOW =
317 // relay on, float = off, so a resetting or unpowered Arduino leaves both relays off.
318 int8_t relayDir = 0;          // +1 = relay 1 (D5) on, -1 = relay 2 (D6) on, 0 = both off
319 uint32_t relayChangeMs = 0;
320
321 // force = switch now (used for stops); otherwise changes are spaced out so the relays
322 // never chatter, and a reversal always passes through "both off" for STEER_RELAY_GAP_MS.
323 void steerRelays(int8_t dir, uint32_t nowMs, bool force) {
324   if (!force) {
325     if (relayDir != 0 && dir != 0 && dir != relayDir) dir = 0;   // reverse via off
326     if (dir == relayDir) return;
327     uint16_t wait = relayDir == 0 ? STEER_RELAY_GAP_MS : STEER_RELAY_MIN_MS;
328     if (nowMs - relayChangeMs < wait) return;
329   } else if (dir == relayDir) {
330     return;
331   }
332   setActiveLow(PIN_STEER_RPWM, dir > 0);
333   setActiveLow(PIN_STEER_LPWM, dir < 0);
334   relayDir = dir;
335   relayChangeMs = nowMs;
336 }
337
338 void steerOff() {
339   if (STEER_RELAYS) {
340     steerRelays(0, millis(), true);
341   } else {
342     analogWrite(PIN_STEER_RPWM, 0);
343     analogWrite(PIN_STEER_LPWM, 0);
344   }
345   digitalWrite(PIN_STEER_ENABLE, LOW);
346   // The stall timer only runs while the motor is driven.
347   stallRefPot = potRaw;
348   stallRefMs = millis();
349 }
350
351 // Drive the motor; pwm > 0 means RPWM. Stops (and holds off for a while) if the
352 // pot does not move although the motor is driven: the gearbox is at an end stop
353 // or jammed, and a toy motor burns out quickly when stalled.
354 void steerMotorRaw(int16_t pwm, uint32_t nowMs) {
355   if (pwm == 0) {
356     steerOff();
357     return;
358   }
359   if (abs((int16_t)potRaw - (int16_t)stallRefPot) > 5) {
360     stallRefPot = potRaw;
361     stallRefMs = nowMs;
362   } else if (nowMs - stallRefMs > STEER_STALL_MS) {
363     steerFaultUntil = nowMs + STEER_FAULT_HOLD_MS;
364     steerState = "STALL";
365     steerOff();
366     return;
367   }
368   digitalWrite(PIN_STEER_ENABLE, HIGH);
369   if (STEER_RELAYS) {
370     int8_t dir = pwm >= STEER_RELAY_ON_PWM ? 1 : (pwm <= -STEER_RELAY_ON_PWM ? -1 : 0);
371     steerRelays(dir, nowMs, dir == 0);
372   } else {
373     analogWrite(PIN_STEER_RPWM, pwm > 0 ? pwm : 0);
374     analogWrite(PIN_STEER_LPWM, pwm < 0 ? -pwm : 0);
```

```
  }
}

// pwm > 0 turns the wheels right. Limit switches block further travel.
void steerTowards(int16_t pwm, uint32_t nowMs) {
  if (pwm > 0 && limitRight()) pwm = 0;
  if (pwm < 0 && limitLeft()) pwm = 0;
  steerMotorRaw(cal.invert ? -pwm : pwm, nowMs);
}

int16_t rcSteerTarget(const RcSnapshot &rc) {
  if (!rc.steerFresh) return 0;
  int16_t d = (int16_t)rc.steerUs - (int16_t)RC_STEER_CENTER_US;
  if (abs(d) < (int16_t)RC_STEER_DEADBAND_US) return 0;
  return (int16_t)constrain(1000L * d / (long)RC_STEER_SPAN_US, -1000L, 1000L);
}

void updateSteering(Mode mode, const RcSnapshot &rc, bool hostOk, uint32_t nowMs) {
  if (nowMs - lastSteerMs < 10) return;
  lastSteerMs = nowMs;
  potRaw = readPot();
  steerPos = calOk ? potToPos(potRaw) : 0;

  if (!STEERING_INSTALLED) {
    steerState = "NONE";
    steerOff();
    return;
  }
  if (mode == MODE_AUTO && hostOk && hostSteerRaw) {   // calibration tool
    steerState = "RAW";
    if (nowMs < steerFaultUntil) { steerState = "STALL"; steerOff(); return; }
    steerMotorRaw(rawSteer, nowMs);
    return;
  }
  if (!calOk) { steerState = "NOCAL"; steerOff(); return; }
  if (potRaw < POT_MIN_VALID || potRaw > POT_MAX_VALID) { steerState = "POT"; steerOff(); return; }
  if (nowMs < steerFaultUntil) { steerState = "STALL"; steerOff(); return; }
  if (mode == MODE_NO_RC) { steerState = "OFF"; steerOff(); return; }

  int16_t target = mode == MODE_MANUAL ? rcSteerTarget(rc) : (hostOk ? hostSteer : 0);
  steerTarget = constrain(target, -STEER_TARGET_LIMIT, STEER_TARGET_LIMIT);
  steerState = "OK";

  int16_t err = steerTarget - steerPos;
  int16_t pwm = 0;
  if (abs(err) > STEER_DEADBAND) {
    int16_t mag = abs(err) >= STEER_SLOW_ZONE
        ? STEER_PWM_MAX
        : STEER_PWM_MIN + (int16_t)((long)(STEER_PWM_MAX - STEER_PWM_MIN) * abs(err) / STEER_SLOW_ZONE);
    pwm = err > 0 ? mag : -mag;
  }
  steerTowards(pwm, nowMs);
}

// ---------------- Serial ----------------

uint8_t parseInts(const char *s, long *out, uint8_t maxCount) {
  uint8_t n = 0;
  while (n < maxCount) {
    char *end;
    out[n] = strtol(s, &end, 10);
    if (end == s) return 0;
    n++;
    s = end;
    if (*s == '\0') return n;
    if (*s != ',') return 0;
    s++;
  }
  return *s == '\0' ? n : 0;
}

void printConfig() {
  Serial.print(F("CFG,pwm_start=")); Serial.print(PWM_START);
  Serial.print(F(",pwm_max=")); Serial.print(PWM_MAX);
  Serial.print(F(",raw_limit=")); Serial.print(PWM_RAW_LIMIT);
  Serial.print(F(",require_rc=")); Serial.print(REQUIRE_RC);
  Serial.print(F(",host_timeout_ms=")); Serial.print(HOST_TIMEOUT_MS);
  Serial.print(F(",steering=")); Serial.print(STEERING_INSTALLED);
  Serial.print(F(",steer_relays=")); Serial.print(STEER_RELAYS);
  Serial.print(F(",cal_ok=")); Serial.print(calOk);
  Serial.print(F(",cal_left=")); Serial.print(cal.left);
  Serial.print(F(",cal_center=")); Serial.print(cal.center);
  Serial.print(F(",cal_right=")); Serial.print(cal.right);
  Serial.print(F(",cal_invert=")); Serial.println(cal.invert);
}

void handleLine(const char *s) {
  long v[4];
  uint8_t n;
  if (s[0] == 'D' && s[1] == ',' && (n = parseInts(s + 2, v, 4)) >= 3) {
    hostLeft = constrain(v[0], 0, 1000);
    hostRight = constrain(v[1], 0, 1000);
    hostBrake = v[2] != 0;
    hostSteer = n == 4 ? constrain(v[3], -1000, 1000) : 0;
    hostRaw = false;
    hostSteerRaw = false;
```

```
    hostCmdMs = millis();
    hostSeen = true;
  } else if (s[0] == 'R' && s[1] == ',' && parseInts(s + 2, v, 2) == 2) {
    rawLeft = constrain(v[0], 0, PWM_RAW_LIMIT);
    rawRight = constrain(v[1], 0, PWM_RAW_LIMIT);
    hostRaw = true;
    hostSteerRaw = false;
    hostCmdMs = millis();
    hostSeen = true;
  } else if (s[0] == 'M' && s[1] == ',' && parseInts(s + 2, v, 1) == 1) {
    rawSteer = constrain(v[0], -RAW_STEER_LIMIT, RAW_STEER_LIMIT);
    hostSteerRaw = true;
    hostRaw = false;
    hostLeft = hostRight = 0;    // car held still while calibrating
    hostBrake = true;
    hostCmdMs = millis();
    hostSeen = true;
  } else if (s[0] == 'K' && s[1] == ',') {
    char which = s[2];
    if (which == 'L') { cal.left = potRaw; saveCal(); }
    else if (which == 'C') { cal.center = potRaw; saveCal(); }
    else if (which == 'R') { cal.right = potRaw; saveCal(); }
    else if (which == 'I' && s[3] == ',') { cal.invert = s[4] == '1'; saveCal(); }
    else if (which == 'X') { cal.magic = 0; EEPROM.put(CAL_ADDR, cal); loadCal(); }
    printConfig();
  } else if (s[0] == '?') {
    printConfig();
  }
}

void readSerial() {
  while (Serial.available()) {
    char c = Serial.read();
    if (c == '\n' || c == '\r') {
      if (lineLen > 0) {
        lineBuf[lineLen] = '\0';
        handleLine(lineBuf);
        lineLen = 0;
      }
    } else if (lineLen < sizeof(lineBuf) - 1) {
      lineBuf[lineLen++] = c;
    } else {
      lineLen = 0;  // overlong line: drop it
    }
  }
}

// Speed drops take effect at once; rises are limited to RAMP_UP_PER_10MS.
void rampTowards(int16_t targetLeft, int16_t targetRight, uint32_t nowMs) {
  if (targetLeft < currentLeft) currentLeft = targetLeft;
  if (targetRight < currentRight) currentRight = targetRight;
  while (nowMs - lastRampMs >= 10) {
    lastRampMs += 10;
    currentLeft = min(targetLeft, (int16_t)(currentLeft + RAMP_UP_PER_10MS));
    currentRight = min(targetRight, (int16_t)(currentRight + RAMP_UP_PER_10MS));
  }
}

const char *modeName(Mode mode) {
  switch (mode) {
    case MODE_MANUAL: return "MANUAL";
    case MODE_AUTO: return "AUTO";
    default: return "NO_RC";
  }
}

void updateLed(Mode mode, uint32_t nowMs) {
  bool on;
  if (mode == MODE_AUTO) on = true;
  else if (mode == MODE_MANUAL) on = (nowMs / 500) % 2;
  else on = (nowMs / 100) % 2;
  digitalWrite(PIN_LED, on);
}

void setup() {
  // Throttles to zero first: the controllers refuse to start with throttle applied.
  pinMode(PIN_THROTTLE_LEFT, OUTPUT);
  pinMode(PIN_THROTTLE_RIGHT, OUTPUT);
  analogWrite(PIN_THROTTLE_LEFT, PWM_IDLE);
  analogWrite(PIN_THROTTLE_RIGHT, PWM_IDLE);
  setActiveLow(PIN_BRAKE_LEFT, true);
  setActiveLow(PIN_BRAKE_RIGHT, true);
  setActiveLow(PIN_DIR_LEFT, INVERT_LEFT);
  setActiveLow(PIN_DIR_RIGHT, INVERT_RIGHT);
  pinMode(PIN_LED, OUTPUT);

  if (!STEER_RELAYS) {         // relay inputs stay floating (= off) until used
    pinMode(PIN_STEER_RPWM, OUTPUT);
    pinMode(PIN_STEER_LPWM, OUTPUT);
  }
  pinMode(PIN_STEER_ENABLE, OUTPUT);
  steerOff();
  pinMode(PIN_LIMIT_LEFT, INPUT_PULLUP);
  pinMode(PIN_LIMIT_RIGHT, INPUT_PULLUP);
  loadCal();

```

```
  pinMode(PIN_RC_THROTTLE, INPUT);
  pinMode(PIN_RC_MODE, INPUT);
  pinMode(PIN_RC_STEER, INPUT);
  attachInterrupt(digitalPinToInterrupt(PIN_RC_THROTTLE), isrThrottle, CHANGE);
  attachInterrupt(digitalPinToInterrupt(PIN_RC_MODE), isrMode, CHANGE);
  PCICR |= _BV(PCIE2);         // pin-change interrupts for D0-D7...
  PCMSK2 = _BV(PCINT20);       // ...but only D4 (CH1)

  Serial.begin(250000);
  Serial.println(F("NAVYA READY v2"));
  printConfig();
  lastRampMs = millis();
}

void loop() {
  readSerial();
  uint32_t nowMs = millis();
  RcSnapshot rc = readRc(nowMs);
  Mode mode = decideMode(rc);
  bool hostOk = hostSeen && nowMs - hostCmdMs < HOST_TIMEOUT_MS;

  int16_t targetLeft = 0;
  int16_t targetRight = 0;
  bool brake = true;
  bool raw = false;

  if (mode == MODE_MANUAL) {
    if (rc.throttleUs >= RC_FORWARD_FROM_US) {
      uint16_t us = constrain(rc.throttleUs, RC_FORWARD_FROM_US, RC_FORWARD_FULL_US);
      targetLeft = targetRight = map(us, RC_FORWARD_FROM_US, RC_FORWARD_FULL_US, 0, 1000);
      brake = false;
    } else if (rc.throttleUs > RC_STOP_BELOW_US) {
      brake = false;  // stick centred: coast
    }
  } else if (mode == MODE_AUTO) {
    bool operatorStop = rc.fresh && rc.throttleUs < RC_STOP_BELOW_US;
    if (!operatorStop && hostOk && !hostSteerRaw) {
      if (hostRaw) {
        raw = true;
        brake = false;
      } else {
        targetLeft = hostLeft;
        targetRight = hostRight;
        brake = hostBrake;
      }
    }
  }

  if (brake) targetLeft = targetRight = 0;

  if (raw) {
    currentLeft = currentRight = 0;
    lastRampMs = nowMs;
    outPwmLeft = rawLeft;
    outPwmRight = rawRight;
  } else {
    rampTowards(targetLeft, targetRight, nowMs);
    outPwmLeft = speedToPwm(currentLeft);
    outPwmRight = speedToPwm(currentRight);
  }
  outBrake = brake;

  analogWrite(PIN_THROTTLE_LEFT, outPwmLeft);
  analogWrite(PIN_THROTTLE_RIGHT, outPwmRight);
  setActiveLow(PIN_BRAKE_LEFT, outBrake);
  setActiveLow(PIN_BRAKE_RIGHT, outBrake);
  updateSteering(mode, rc, hostOk, nowMs);
  updateLed(mode, nowMs);

  if (nowMs - lastStatusMs >= STATUS_PERIOD_MS) {
    lastStatusMs = nowMs;
    Serial.print(F("S,"));
    Serial.print(modeName(mode)); Serial.print(',');
    Serial.print(rc.throttleUs); Serial.print(',');
    Serial.print(rc.modeUs); Serial.print(',');
    Serial.print(hostOk); Serial.print(',');
    Serial.print(outPwmLeft); Serial.print(',');
    Serial.print(outPwmRight); Serial.print(',');
    Serial.print(outBrake); Serial.print(',');
    Serial.print(rc.steerUs); Serial.print(',');
    Serial.print(steerTarget); Serial.print(',');
    Serial.print(steerPos); Serial.print(',');
    Serial.print(potRaw); Serial.print(',');
    Serial.print(limitLeft()); Serial.print(',');
    Serial.print(limitRight()); Serial.print(',');
    Serial.print(steerState); Serial.print(',');
    Serial.print(batteryMv(PIN_BATT_LEFT)); Serial.print(',');
    Serial.println(batteryMv(PIN_BATT_RIGHT));
  }
}
```